

МИНИСТЕРСТВО ТРАНСПОРТА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ АГЕНТСТВО ЖЕЛЕЗНОДОРОЖНОГО ТРАНСПОРТА
Государственное бюджетное образовательное учреждение
высшего образования
**«ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПУТЕЙ СООБЩЕНИЯ ИМПЕРАТОРА АЛЕКСАНДРА I»**

Кафедра «ИНФОРМАЦИОННЫЕ И ВЫЧИСЛИТЕЛЬНЫЕ СИСТЕМЫ»

Дисциплина: «Программирование Ассемблер»

О Т Ч Е Т
по лабораторной работе № 2.3

Выполнил студент
Факультета АИТ
Группы ИВБ-515
Принял

Нартов С. А.

Кукин М. Ю.

Санкт-Петербург
2026

Задача 1

На основе ранее выполненной работы 2.2 написать программу, осуществляющие расчет по тому же варианту формул с использованием математического сопроцессора и чисел с плавающей точкой.

Листинг

.686p

.model flat, stdcall

option casemap: none

```
include C:\masm32\include\windows.inc
include C:\masm32\include\kernel32.inc
includelib C:\masm32\lib\kernel32.lib
include C:\masm32\include\user32.inc
includelib C:\masm32\lib\user32.lib
include C:\masm32\include\masm32.inc
includelib C:\masm32\lib\masm32.lib
include C:\masm32\include\msvcrt.inc
includelib C:\masm32\lib\msvcrt.lib
include C:\masm32\macros\macros.asm
```

.data

```
szAppName db "Nartov Sergey IVB-515 LR2.3",0
messagestart db "Please, enter X: ",0
messageresult db "Your result is: %lf",13,10,0
messagebadnum db "Your number is out of range or you've got division by
zero",13,10,0
inputs db "%lf",0

three dq 3.0
one dq 1.0
neg20 dq -20.0
fifty dq 50.0
eighteen dq 18.0
```

.data?

```
x dq ?
y dq ?
```

.code

main PROC

 finit

 invoke SetConsoleTitleA, Addr szAppName

 invoke crt_printf, addr messagestart

 invoke crt_scanf, addr inputs, addr x

 ; Проверяем x >= -20

 fld x ; st(0) = x

 fld neg20 ; st(0) = -20, st(1) = x

 fcomip st(0), st(1) ; сравниваем -20 и x (st(0) с st(1))

```
ja check_range_failed ; если  $-20 > x$  (т.е.  $x < -20$ ) то ошибка
```

```
; Проверяем  $x \leq 50$ 
```

```
fld x ; st(0) = x
```

```
fld fifty ; st(0) = 50, st(1) = x
```

```
fcomip st(0), st(1) ; сравниваем 50 и x
```

```
jnb check_range_failed ; если  $50 < x$  (т.е.  $x > 50$ ) то ошибка
```

```
; ПРОВЕРКА УСЛОВИЯ  $x < 18$ 
```

```
fld x ; st(0) = x
```

```
fld eighteen ; st(0) = 18, st(1) = x
```

```
fcomip st(0), st(1) ; сравниваем 18 и x
```

```
ja case_two ; если  $18 > x$  (т.е.  $x < 18$ ) то case_two
```

```
jmp case_four ; иначе case_four
```

```
check_range_failed:
```

```
jmp bad_range
```

```
case_two:
```

```
;  $y = (x+3)^2 / (1-x)$ 
```

```
; Проверка на деление на ноль ( $x = 1$ )
```

```
fld x
```

```
fld one
```

```
fcomip st(0), st(1) ; сравниваем x и 1
```

```
je bad_range ; если  $x = 1$ , то ошибка
```

```
; Вычисляем  $(x+3)^2$ 
```

```
fld x
```

```
fld three
```

```
faddp st(1), st(0) ; st(0) = x+3
```

```
fld st(0)
```

```
fmlp st(1), st(0) ; st(0) =  $(x+3)^2$ 
```

```
; Вычисляем  $(1-x)$ 
```

```
fld one
```

```
fld x
```

```
fsubp st(1), st(0) ; st(0) =  $1-x$ 
```

```
; Делим
```

```
fdivp st(1), st(0) ; st(0) =  $(x+3)^2 / (1-x)$ 
```

```
fstp y
```

```
invoke crt_printf, addr messengeresult, y
```

```
jmp EndProgram
```

```
case_four:
```

```
;  $y = (x+3)^3$ 
```

```
fld x
```

```
fld three
```

```
faddp st(1), st(0) ; st(0) = x+3
```

```
fld st(0)
fmulp st(1), st(0)    ; st(0) = (x+3)^2

fld x
fld three
faddp st(1), st(0)    ; st(0) = x+3
fmulp st(1), st(0)    ; st(0) = (x+3)^3

fstp y

invoke crt_printf, addr messageresult, y
jmp EndProgram

bad_range:
    invoke crt_printf, addr messagebadnum

EndProgram:
    inkey "Press any key for end..."
    invoke ExitProcess, 0

main ENDP
end main
```

Отладка

Задача 2

Изучите следующие сведения о движении материальной точки в поле силы тяжести ($g=9,8\text{м/с}^2$):

Листинг

```

.686p
.model flat, stdcall
option casemap: none

include C:\masm32\include\windows.inc
include C:\masm32\include\kernel32.inc
includelib C:\masm32\lib\kernel32.lib
include C:\masm32\include\user32.inc
includelib C:\masm32\lib\user32.lib
include C:\masm32\include\masm32.inc
includelib C:\masm32\lib\masm32.lib
include C:\masm32\include\msvcrt.inc
includelib C:\masm32\lib\msvcrt.lib
include C:\masm32\macros\macros.asm

.data
    szPromptSpeed    db "Enter initial speed: ", 0
    szPromptHeight    db "Enter maximum height: ", 0
    szError           db "Erroe...", 13, 10, 0

    szFmtTime         db "Flight time: %lf s", 13, 10, 0
    szFmtRange         db "Flight range: %lf m", 13, 10, 0
    szFmtSpeed         db "Speed: %6lf", 13, 10, 0
    szFmtHeight        db "Height: %lf", 13, 10, 0

    szWindowTitle     db "Nartov Sergey IVB-515 LR2.3", 0

    szInputFormat      db "%6lf", 0

    gravity            dq 9.8          ; ускорение свободного падения (м/с²)
    const_one          dq 1.0          ; константа 1.0
    const_two          dq 2.0          ; константа 2.0

.data?
    hConsoleOutput     HANDLE ?       ; дескриптор для вывода
    hConsoleInput      HANDLE ?       ; дескриптор для ввода

    sin_alpha          dq ?           ; синус угла
    cos_alpha          dq ?           ; косинус угла

    initial_speed       dq ?           ; начальная скорость v0
    max_height          dq ?           ; максимальная высота H

    flight_time         dq ?           ; время полёта t
    flight_range        dq ?           ; дальность полёта L

.code

```

main PROC

```
invoke GetStdHandle, STD_OUTPUT_HANDLE
mov hConsoleOutput, eax
```

```
invoke GetStdHandle, STD_INPUT_HANDLE
mov hConsoleInput, eax
```

```
; starter speed
invoke crt_printf, addr szPromptSpeed
invoke crt_scanf, addr szInputFormat, addr initial_speed
```

```
; Enter max Height
invoke crt_printf, addr szPromptHeight
invoke crt_scanf, addr szInputFormat, addr max_height
```

```
;  $\sin\alpha = \sqrt{2gH} / (v_0)$ 
;  $\sin\alpha = \sqrt{2 * g * H} / v_0$ 
```

```
;-----
```

```
fld    gravity           ; st(0) = g
fld    max_height        ; st(0) = H, st(1) = g
fmul   const_two         ; st(0) = 2*H, st(1) = g
fmulp  st(1), st(0)       ; st(0) = 2*g*H
fsqrt                    ; st(0) =  $\sqrt{2*g*H}$ 
fld    initial_speed     ; st(0) =  $v_0$ , st(1) =  $\sqrt{2*g*H}$ 
fdivp  st(1), st(0)       ; st(0) =  $\sqrt{2*g*H}/v_0 = \sin\alpha$ 
```

```
;  $\sin\alpha$  must be  $\leq 1$ 
fld    const_one         ; st(0) = 1, st(1) =  $\sin\alpha$ 
fcomip st(0), st(1)       ; сравнение 1 и  $\sin\alpha$ 
jb     error_occurred    ; если  $\sin\alpha > 1 \rightarrow$  ошибка
```

```
fstp   sin_alpha         ; сохраняем  $\sin\alpha$ 
```

```
;  $\cos\alpha = \sqrt{1 - \sin^2\alpha}$ 
fld    const_one         ; st(0) = 1
fld    sin_alpha         ; st(0) =  $\sin\alpha$ , st(1) = 1
fld    st(0)             ; st(0) =  $\sin\alpha$ , st(1) =  $\sin\alpha$ , st(2) = 1
fmulp  st(1), st(0)       ; st(0) =  $\sin^2\alpha$ , st(1) = 1
fsubp  st(1), st(0)       ; st(0) =  $1 - \sin^2\alpha$ 
fsqrt                    ; st(0) =  $\sqrt{1 - \sin^2\alpha} = \cos\alpha$ 
fstp   cos_alpha         ; сохраняем  $\cos\alpha$ 
```

```
; Вычисление времени полёта:
```

```
;  $t = 2 * v_0 * \sin\alpha / g$ 
fld    const_two         ; st(0) = 2
fld    initial_speed     ; st(0) =  $v_0$ , st(1) = 2
fmul   sin_alpha         ; st(0) =  $v_0*\sin\alpha$ , st(1) = 2
fmulp  st(1), st(0)       ; st(0) =  $2*v_0*\sin\alpha$ 
fdiv   gravity           ; st(0) =  $2*v_0*\sin\alpha/g = t$ 
fstp   flight_time       ; сохраняем t
```

```

invoke crt_printf, addr szFmtTime, flight_time

; Вычисление дальности полёта:
;       $L = 2 * v_0^2 * \sin\alpha * \cos\alpha / g$ 
fld     sin_alpha           ; st(0) = sin $\alpha$ 
fld     cos_alpha           ; st(0) = cos $\alpha$ , st(1) = sin $\alpha$ 
fmulp   st(1), st(0)         ; st(0) = sin $\alpha$ *cos $\alpha$ 

fld     initial_speed       ; st(0) =  $v_0$ , st(1) = sin $\alpha$ *cos $\alpha$ 
fld     st(0)                ; st(0) =  $v_0$ , st(1) =  $v_0$ , st(2) = sin $\alpha$ *cos $\alpha$ 
fmulp   st(1), st(0)         ; st(0) =  $v_0^2$ , st(1) = sin $\alpha$ *cos $\alpha$ 
fmulp   st(1), st(0)         ; st(0) =  $v_0^2$ *sin $\alpha$ *cos $\alpha$ 

fld     const_two           ; st(0) = 2, st(1) =  $v_0^2$ *sin $\alpha$ *cos $\alpha$ 
fmulp   st(1), st(0)         ; st(0) =  $2*v_0^2$ *sin $\alpha$ *cos $\alpha$ 
fddiv   gravity              ; st(0) =  $2*v_0^2$ *sin $\alpha$ *cos $\alpha$ /g = L
fstp    flight_range        ; сохраняем L

invoke crt_printf, addr szFmtRange, flight_range
jmp     finish_program

error_occurred:
    invoke crt_printf, addr szError

finish_program:
    inkey "Press any key for exit..."
    invoke ExitProcess, 0
main ENDP
END main

```

Вывод

Я научился проводить математические операции над числами с плавающей точкой в языке masn32